

Aufgabenstellung: Objektorientiertes Design in Java

(SOLID-Prinzipien mit Enums, Assoziationen, Vererbung, abstrakten Klassen und Interfaces)

Ziel der Aufgabe

In dieser Aufgabe analysieren und entwickeln Sie ein Java-Programm, das eine objektorientierte Modellierung für ein **Medien- und Ausleihsystem** beschreibt.

Sie lernen dabei,

- wie die **SOLID-Prinzipien** in Java praktisch umgesetzt werden,
- wie **Enums** zur Modellierung fester Zustände eingesetzt werden,
- wie **Assoziationen** zwischen Klassen fachlich sinnvoll beschrieben werden,
- wie **Vererbung, abstrakte Klassen** und **Polymorphie** zusammenwirken,
- wie **Interfaces** zur Entkopplung und Erweiterbarkeit beitragen,
- wie Beziehungen und Hierarchien in einem **Klassendiagramm** dargestellt werden.

Der Fokus liegt auf dem Verständnis, dass gutes objektorientiertes Design nicht nur aus funktionierendem Code besteht, sondern aus einer **klaren, erweiterbaren und wartbaren Struktur**.

Fachlicher Hintergrund

In der objektorientierten Programmierung entstehen häufig Modelle, in denen verschiedene Objekte zusammenarbeiten.

In dieser Aufgabe geht es um ein kleines Bibliotheks- bzw. Mediensystem:

- eine Bibliothek verwaltet Medien,
- ein Medium kann ausgeliehen werden,
- ein Buch ist ein Medium,
- eine DVD ist ein Medium,
- eine Person kann Medien ausleihen,
- bestimmte Medien können gedruckt, archiviert oder digital bereitgestellt werden.

Zur sauberen Modellierung sollen dabei gezielt verschiedene Konzepte verbunden werden:

- **Enums** zur Beschreibung fester Zustände,

- **Assoziationen** zur Modellierung von Beziehungen,
- **Vererbung** zur Spezialisierung,
- **abstrakte Klassen** zur Bündelung gemeinsamer Merkmale,
- **Interfaces** zur Beschreibung gemeinsamer Fähigkeiten,
- **SOLID-Prinzipien** zur Begründung der Entwurfsentscheidungen.

Die fünf SOLID-Prinzipien lauten:

- **S** – Single Responsibility Principle
- **O** – Open/Closed Principle
- **L** – Liskov Substitution Principle
- **I** – Interface Segregation Principle
- **D** – Dependency Inversion Principle

Damit eignet sich diese Aufgabe gut, um zentrale Inhalte der objektorientierten Programmierung in einem zusammenhängenden Beispiel zu wiederholen.

Vorgegebene Klassenstruktur

Im Package `solid.medien` sollen folgende Klassen, Interfaces und

Enums entwickelt und verwendet werden:

- Medium
- Buch
- DVD
- Person
- Bibliothek
- Ausleihe
- Medienstatus
- Ausleihbar
- Druckbar
- Archivierbar
- Main

Die Elemente haben dabei folgende Rollen:

- Medium ist die abstrakte Oberklasse,
- Buch und DVD sind konkrete Unterklassen von Medium,
- Person beschreibt Benutzer des Systems,
- Bibliothek verwaltet Medien,
- Ausleihe modelliert eine Beziehung zwischen Person und Medium,
- Medienstatus ist ein Enum,

- Ausleihbar, Druckbar und Archivierbar sind Interfaces,
- Main enthält die main-Methode zum Testen des Programms.

Aufgabe 1: Entwicklung des Enums „Medienstatus“

Ziel

Sie entwickeln das Enum Medienstatus, um feste Zustände eines Mediums zu modellieren.

Zu entwickelnde Struktur

Das Enum soll mindestens folgende Werte enthalten:

VERFUEGBAR
AUSGELIEHEN
RESERVIERT
BESCHAEDIGT

Zusätzlich soll das Enum erweitert werden, zum Beispiel durch:

```
private String anzeigeText;  
Medienstatus(String anzeigeText)  
public String getAnzeigeText()
```

Außerdem soll das Enum später in einer `switch`-Anweisung verwendet werden.

Anforderungen

Funktionale Anforderungen

Entwickeln Sie das Enum Medienstatus so, dass:

- feste Zustände eines Mediums modelliert werden,
- jeder Zustand eine verständliche textuelle Beschreibung besitzen kann,
- Zustände im Programm ausgegeben und unterschieden werden können.

Technische Anforderungen

Verwenden Sie:

- das Schlüsselwort `enum`,
- mindestens vier Enum-Werte,
- mindestens ein Feld,
- einen Konstruktor,
- mindestens eine Methode,
- eine spätere Verwendung in einer `switch`-Anweisung.

SOLID-Bezug

S – Single Responsibility Principle

Das Enum hat genau eine klar abgegrenzte Aufgabe: die Modellierung von Zuständen.

O – Open/Closed Principle

Das System kann durch zusätzliche Enum-Werte erweitert werden, ohne dass die Grundidee der Modellierung verändert werden muss.

Ziel dieser Aufgabe

Sie sollen verstehen,

- was ein Enum in Java ist,
- wie Enums mit Feldern, Methoden und Konstruktoren erweitert werden,
- warum Enums besser geeignet sind als lose Strings für feste Zustände.

Aufgabe 2: Entwicklung der abstrakten Klasse „Medium“

Ziel

Sie entwickeln die Klasse Medium als abstrakte Oberklasse.

Zu entwickelnde Klassenstruktur

Die Klasse Medium soll gemeinsame Merkmale aller Medien enthalten.

Beispielhaft kann die Klasse mindestens enthalten:

```
private String titel;  
private Medienstatus status;
```

sowie passende Methoden, zum Beispiel:

```
public Medium(String titel, Medienstatus status)  
public String getTitel()  
public Medienstatus getStatus()  
public void setStatus(Medienstatus status)  
public String beschreibe()  
public abstract String gibMedienInfo()
```

Anforderungen

Funktionale Anforderungen

Entwickeln Sie die Klasse `Medium` so, dass:

- jedes `Medium` einen Titel besitzt,
- jedes `Medium` einen Status besitzt,
- gemeinsame Eigenschaften in der Oberklasse gebündelt werden,
- mindestens eine gemeinsame konkrete Methode vorhanden ist,
- mindestens eine abstrakte Methode für Unterklassen vorgegeben wird.

Technische Anforderungen

Verwenden Sie:

- das Schlüsselwort `abstract`,
- mindestens zwei Attribute,
- einen Konstruktor,
- Getter-Methoden,
- mindestens eine konkrete Methode,
- mindestens eine abstrakte Methode.

SOLID-Bezug

S – Single Responsibility Principle

Medium soll nur das gemeinsame Grundmodell aller Medien beschreiben.

L – Liskov Substitution Principle

Unterklassen müssen überall dort sinnvoll einsetzbar sein, wo ein Medium erwartet wird.

Ziel dieser Aufgabe

Sie sollen verstehen,

- wie abstrakte Klassen gemeinsame Eigenschaften bündeln,
- warum eine Oberklasse nicht alle Spezialfälle selbst enthalten sollte,
- wie eine saubere Vererbungsstruktur vorbereitet wird.

Aufgabe 3: Entwicklung der Klassen „Buch“ und „DVD“

Ziel

Sie entwickeln die konkreten Unterklassen Buch und DVD.

Zu entwickelnde Klassenstruktur

```
public class Buch extends Medium
public class DVD extends Medium
```

Anforderungen

Funktionale Anforderungen

Entwickeln Sie die Klassen so, dass:

- beide Klassen von Medium erben,
- beide die abstrakte Methode der Oberklasse konkret umsetzen,
- beide zusätzliche eigene Merkmale besitzen können,
- sich beide in mindestens einer fachlichen Eigenschaft unterscheiden.

Technische Anforderungen

Verwenden Sie:

- `extends Medium`,
- Konstruktoren mit `super(...)`,

- @Override,
- mindestens eine eigene Implementierung der abstrakten Methode.

SOLID-Bezug

L – Liskov Substitution Principle

Buch und DVD müssen sich fachlich korrekt wie Medium verhalten.

O – Open/Closed Principle

Neue Medientypen wie Zeitschrift oder EBook sollen später ergänzbar sein, ohne bestehende Oberklassenlogik umzubauen.

Ziel dieser Aufgabe

Sie sollen verstehen,

- wie konkrete Unterklassen aus einer abstrakten Oberklasse entstehen,
- wie `super( . . . )` zur Initialisierung geerbter Merkmale verwendet wird,
- wie Vererbung Spezialisierung ermöglicht.

Aufgabe 4: Analyse von Polymorphie

Ziel

Sie analysieren, wie Objekte verschiedener Unterklassen über den Oberklassen-Typ behandelt werden können.

Anforderungen

Funktionale Anforderungen

Untersuchen und entwickeln Sie die Klassen so, dass:

- Objekte von Buch und DVD über den Typ Medium angesprochen werden können,
- Methodenaufrufe je nach tatsächlicher Unterklasse unterschiedlich wirken,
- Polymorphie in der Ausgabe sichtbar wird.

Technische Anforderungen

Beziehen Sie sich auf:

- Oberklassen-Referenzen,

- Unterklassen-Objekte,
 - überschriebenen Methoden,
 - dynamische Bindung.
-

SOLID-Bezug

L – Liskov Substitution Principle

Die Unterklassen müssen ohne Fehler und ohne inhaltliche Widersprüche als Medium einsetzbar sein.

D – Dependency Inversion Principle

Arbeiten mit dem abstrakten Typ Medium statt nur mit konkreten Klassen fördert lose Kopplung.

Ziel dieser Aufgabe

Sie sollen verstehen,

- was Polymorphie bedeutet,
- wie Methodenaufrufe zur Laufzeit gebunden werden,
- warum Polymorphie für flexible Programme wichtig ist.

Aufgabe 5: Entwicklung des Interfaces „Ausleihbar“

Ziel

Sie entwickeln das Interface `Ausleihbar`.

Zu entwickelnde Struktur

Das Interface kann mindestens enthalten:

```
void ausleihen(Person person);  
void zurueckgeben();
```

Anforderungen

Funktionale Anforderungen

Entwickeln Sie das Interface so, dass:

- ausleihbare Objekte ein einheitliches Verhalten vorgeben,
- verschiedene Medientypen diese Vorgabe umsetzen können.

Technische Anforderungen

Verwenden Sie:

- das Schlüsselwort `interface`,
 - mindestens zwei abstrakte Methoden,
 - eine Implementierung in mindestens einer konkreten Klasse.
-

SOLID-Bezug

I – Interface Segregation Principle

Das Interface soll nur Verhalten enthalten, das wirklich zu ausleihbaren Objekten passt.

D – Dependency Inversion Principle

Andere Klassen sollen gegen das Interface `Ausleihbar` arbeiten können statt gegen eine konkrete Klasse.

Ziel dieser Aufgabe

Sie sollen verstehen,

- wie Interfaces gemeinsame Verträge festlegen,
- wie sich Interfaces von abstrakten Klassen unterscheiden,

- warum kleine, passende Interfaces vorteilhaft sind.

Aufgabe 6: Entwicklung der Interfaces „Druckbar“ und „Archivierbar“

Ziel

Sie entwickeln weitere Interfaces zur Wiederholung von Interface-Konzepten.

Zu entwickelnde Struktur

Das Interface Druckbar soll enthalten:

- eine abstrakte Methode,
- eine default-Methode,
- eine static-Methode.

Beispielhaft:

```
void drucken();  
default boolean pruefeFormat(String format)
```

```
static void testDruck()
```

Das Interface Archivierbar soll ein eigenes, klar abgegrenztes Verhalten beschreiben.

Anforderungen

Funktionale Anforderungen

Entwickeln Sie die Interfaces so, dass:

- Druckbar nur druckbezogenes Verhalten enthält,
- Archivierbar nur archivierungsbezogenes Verhalten enthält,
- mindestens eine Klasse eines der Interfaces implementiert,
- default- und static-Methoden genutzt werden.

Technische Anforderungen

Verwenden Sie:

- Interface-Deklarationen,
- mindestens eine abstrakte Methode,
- mindestens eine default-Methode,
- mindestens eine static-Methode,
- mindestens eine Implementierung in einer Klasse.

SOLID-Bezug

I – Interface Segregation Principle

Statt eines großen Sammel-Interfaces entstehen mehrere kleine, passende Interfaces.

O – Open/Closed Principle

Neue Fähigkeiten können durch weitere Interfaces ergänzt werden, ohne bestehende Klassenstrukturen grundlegend zu verändern.

Ziel dieser Aufgabe

Sie sollen verstehen,

- wie Interfaces deklariert und implementiert werden,
- worin der Unterschied zwischen Interface und abstrakter Klasse besteht,
- wie default- und static-Methoden in Interfaces eingesetzt werden.

Aufgabe 7: Entwicklung der Klasse „Person“

Ziel

Sie entwickeln die Klasse Person als eigenständige Fachklasse.

Zu entwickelnde Klassenstruktur

Die Klasse kann mindestens enthalten:

```
private String name;
```

sowie passende Methoden, zum Beispiel:

```
public Person(String name)
public String getName()
```

Anforderungen

Funktionale Anforderungen

Entwickeln Sie die Klasse so, dass:

- eine Person einen Namen besitzt,
- eine Person als Entität in Beziehungen zu anderen Klassen verwendet werden kann.

Technische Anforderungen

Verwenden Sie:

- private Attribute,
 - einen Konstruktor,
 - Getter-Methoden.
-

SOLID-Bezug

S – Single Responsibility Principle

Person soll nur die Person selbst modellieren und nicht zusätzliche Medien- oder Bibliothekslogik enthalten.

Ziel dieser Aufgabe

Sie sollen verstehen,

- wie eigenständige Fachobjekte modelliert werden,
 - warum Beziehungen zwischen Klassen oft durch separate Klassen klarer werden.
-

Aufgabe 8: Entwicklung der Klasse „Ausleihe“ und Analyse von Assoziationen

Ziel

Sie entwickeln die Klasse `Ausleihe` und untersuchen verschiedene Arten von Assoziationen.

Zu entwickelnde Klassenstruktur

Die Klasse `Ausleihe` soll Beziehungen zwischen `Person` und `Medium` modellieren.

Beispielhaft:

```
private Person person;  
private Medium medium;
```

Zusätzlich sollen in der Gesamtstruktur folgende Beziehungsarten erkennbar sein:

- unidirektionale Assoziation,
- bidirektionale Assoziation,
- Aggregation,
- Komposition.

Anforderungen

Funktionale Anforderungen

Entwickeln und analysieren Sie die Klassenstruktur so, dass:

- eine Ausleihe genau einer Person und genau einem Medium zugeordnet ist,
- eine Bibliothek mehrere Medien verwalten kann,
- Beziehungen fachlich begründet werden,
- Navigierbarkeit und Multiplizität beschrieben werden können.

Technische Anforderungen

Beziehen Sie sich auf:

- 1:1-, 1:n- und mögliche n:m-Beziehungen,
 - Referenzen zwischen Klassen,
 - Listen für mehrere Objekte,
 - die Unterscheidung von Aggregation und Komposition,
 - Klassendiagramm-Darstellung.
-

SOLID-Bezug

S – Single Responsibility Principle

Die Klasse `Ausleihe` modelliert gezielt die Ausleihbeziehung statt diese Logik in andere Klassen zu mischen.

D – Dependency Inversion Principle

Beziehungen sollen möglichst über geeignete Abstraktionen und klar definierte Verantwortungen modelliert werden.

Ziel dieser Aufgabe

Sie sollen verstehen,

- was Assoziationen zwischen Klassen sind,
- wie sich unidirektionale und bidirektionale Beziehungen unterscheiden,
- was Multiplizität, Navigierbarkeit, Aggregation und Komposition bedeuten.

Aufgabe 9: Entwicklung der Klasse „Bibliothek“

Ziel

Sie entwickeln die Klasse `Bibliothek` als verwaltende Klasse.

Zu entwickelnde Klassenstruktur

Die Klasse `Bibliothek` kann mindestens enthalten:

```
private List<Medium> medien;
```

sowie passende Methoden, zum Beispiel:

```
public void mediumHinzufuegen(Medium medium)
public void alleMedienAusgeben()
```

Außerdem soll eine `switch`-Anweisung auf Basis des Enums `Medienstatus` eingesetzt werden.

Anforderungen

Funktionale Anforderungen

Entwickeln Sie die Klasse `Bibliothek` so, dass:

- mehrere Medien verwaltet werden können,
- Medien über den Typ `Medium` gespeichert werden,

- der Status eines Mediums per switch ausgewertet werden kann,
- die Bibliothek mit unterschiedlichen Medientypen arbeiten kann.

Technische Anforderungen

Verwenden Sie:

- `List<Medium>`,
- Polymorphie,
- eine switch-Anweisung mit Medienstatus,
- mindestens eine Ausgabe, die verschiedene Medien korrekt behandelt.

SOLID-Bezug

D – Dependency Inversion Principle

Die Bibliothek arbeitet mit dem abstrakten Typ `Medium` statt nur mit `Buch` oder `DVD`.

O – Open/Closed Principle

Neue Medientypen sollen ergänzt werden können, ohne die Grundstruktur der Bibliothek grundlegend umzuschreiben.

Ziel dieser Aufgabe

Sie sollen verstehen,

- wie Polymorphie in verwaltenden Klassen genutzt wird,
- wie Enums in switch-Anweisungen eingesetzt werden,
- warum Abhängigkeiten auf abstrakte Typen flexibler sind.

Aufgabe 10: Analyse von Interfaces, abstrakten Klassen und Klassendiagrammen

Ziel

Sie analysieren die gesamte Struktur aus Sicht des objektorientierten Designs.

Anforderungen

Funktionale Anforderungen

Erklären und analysieren Sie,

- warum Medium als abstrakte Klasse geeignet ist,

- warum Ausleihbar, Druckbar und Archivierbar als Interfaces sinnvoll sind,
- worin der Unterschied zwischen abstrakter Klasse und Interface liegt,
- wie Vererbung und Interface-Implementierung im Klassendiagramm dargestellt werden,
- welche SOLID-Prinzipien in der Modellierung sichtbar werden.

Technische Anforderungen

Beziehen Sie sich auf:

- abstrakte Oberklasse und konkrete Unterklassen,
- Interface-Deklarationen und Implementierungen,
- Vererbungsbeziehungen,
- Klassendiagramm mit Assoziationen, Multiplizitäten und Navigierbarkeit,
- SOLID-Prinzipien.

SOLID-Bezug

Hier sollen **alle fünf SOLID-Prinzipien** in der Gesamtstruktur erkannt und erläutert werden:

- **S** – klare Verantwortlichkeiten
 - **O** – Erweiterbarkeit ohne ständiges Umbauen
 - **L** – sinnvolle Ersetzbarkeit von Unterklassen
 - **I** – kleine, passende Interfaces
 - **D** – Abhängigkeiten zu Abstraktionen statt nur zu konkreten Klassen
-

Ziel dieser Aufgabe

Sie sollen verstehen,

- wie verschiedene OOP-Konzepte zusammenwirken,
 - wie man eine Klassenstruktur fachlich und technisch begründet,
 - wie gutes Design anhand von SOLID reflektiert werden kann.
-

Lernziele

Nach Abschluss dieser Aufgabe können Sie:

- erklären, was ein Enum in Java ist,

- Enums mit Feldern, Konstruktoren und Methoden verwenden,
- Enums in switch-Anweisungen einsetzen,
- Assoziationen zwischen Klassen unterscheiden und beschreiben,
- Multiplizität, Navigierbarkeit, Aggregation und Komposition erläutern,
- Vererbung mit `extends` und `super( ... )` anwenden,
- Polymorphie erklären und nutzen,
- abstrakte Klassen von Interfaces unterscheiden,
- Interfaces deklarieren und implementieren,
- `default`- und `static`-Methoden in Interfaces einordnen,
- Beziehungen im Klassendiagramm beschreiben,
- SOLID-Prinzipien in einer Modellierung erkennen und begründen.

Didaktische Einordnung

Diese Aufgabe verbindet mehrere zentrale Themen der objektorientierten Programmierung in einem gemeinsamen Beispiel.

Sie knüpft an folgende Themen an:

- Klassen und Objekte

- Attribute und Methoden
- Konstruktoren
- Vererbung
- `extends`
- `super(...)`
- Polymorphie
- abstrakte Klassen
- Interfaces
- `default-` und `static`-Methoden
- Enums
- Assoziationen
- UML-Klassendiagramme
- objektorientierte Modellierung
- SOLID-Prinzipien

Die Aufgabe zeigt, dass gutes objektorientiertes Design nicht aus isolierten Einzelthemen besteht, sondern aus dem bewussten Zusammenspiel von Modellierung, Abstraktion, Beziehungen und Verantwortlichkeiten.

Reflexions- und Kontrollfragen

Kontrollfragen zu SOLID

1. Was bedeutet das Single Responsibility Principle?
 2. Warum unterstützt eine abstrakte Oberklasse oft das Open/Closed Principle?
 3. Was fordert das Liskov Substitution Principle bei Unterklassen?
 4. Warum passen kleine Interfaces gut zum Interface Segregation Principle?
 5. Warum unterstützt die Arbeit mit abstrakten Typen das Dependency Inversion Principle?
-

Kontrollfragen zu Enums

6. Was ist ein Enum in Java?
 7. Warum sind Enums besser geeignet als frei gewählte Strings für feste Zustände?
 8. Wie können Enums um Felder, Konstruktoren und Methoden erweitert werden?
 9. Wie wird ein Enum in einer `switch`-Anweisung verwendet?
-

Kontrollfragen zu Assoziationen

10. Was ist eine unidirektionale Assoziation?
 11. Was ist eine bidirektionale Assoziation?
 12. Was bedeuten Multiplizität und Navigierbarkeit?
 13. Worin unterscheiden sich Aggregation und Komposition?
 14. Wie werden Assoziationen im Klassendiagramm dargestellt?
-

Kontrollfragen zu Vererbung und Interfaces

15. Warum ist `Medium` eine sinnvolle abstrakte Oberklasse?
 16. Welche Aufgabe hat `super ( . . . )` im Konstruktor?
 17. Was bedeutet Polymorphie?
 18. Worin unterscheiden sich Interfaces und abstrakte Klassen?
 19. Welche Aufgabe haben `default`- und `static`-Methoden in Interfaces?
 20. Wie werden Vererbung und Interface-Implementierung im Klassendiagramm dargestellt?
-